摘　要

在日常生活及工业生产过程中，经常要用到温度的检测及控制，温度是生产过程和科学实验中普遍而且重要的物理参数之一。在生产过程中，为了高效地进行生产，必须对它的主要参数，如温度、压力、流量等进行有效的控制。温度控制在生产过程中占有相当大的比例。温度测量是温度控制的基础，技术已经比较成熟。传统的测温元件有热电偶和二电阻。而热电偶和热电阻测出的一般都是电压，再转换成对应的温度，这些方法相对比较复杂，需要比较多的外部硬件支持。

本文采用美国 DALLAS 半导体公司推出的一种改进型温度传感器 DS18B20 作为检测元件，温度范围为-55～125ºC,最高分辨率可达 0.0625ºC。DS18B20 可以直接读出被测温度值，而且采用三线制与单片机相连，减少了外部的硬件电路，具有低成本和易使用的特点。

本文介绍一种基于 STC89C51 单片机的温度测量及报警电路,该电路采用 DS18B20 作为温度监测元件，测量范围-55℃-～+125℃，使用液晶模块显示，能设置温度报警上下限。正文着重给出了软硬件系统的各部分电路，介绍了集成温度传感器 DS18B20 的原理，STC89C51 单片机功能和应用。该电路设计合理、功能实用、结构简单。

关键词： STC89C52RC 单片机；温度传感器 DS18B20；lcd 液晶显示器

ABSTRACT

In daily life and industrial production process, often used in the detection and control of temperature, temperature is the production process and scientific experiments in general and one of the important physical parameter. In the production process, in order to efficiently carr y out the production, to be its main parameters, such as temperature, pressure, flow control, etc... Temperature measurement is the basis of temperature-controlled, more mature technolo gy.Traditional thermocouple and temperature components are the second resistor.The thermo couple and thermal resistance are generally measured voltage, and then replaced by the corres ponding temperature, these methods are relatively complex, requiring a relatively large numb er of external hardware support.

This article USES the DALLAS semiconductor companies in the United States launched a modified temperature sensor DS18B20 as detecting element, a temperature range of-55~125º C, up to a maximum resolution of 0.0625 º C. DS18B20 can be directly read out t he temperature on the north side, and three-wire system with single-chip connected to a decre ase of the external hardware circuit, with low-cost and easy use.

The introduction of a cost-based STC89C51 MCU temperatur measurement circuits, the circuits used DS18B20 high-precision temperatur sensor, measuring scope -55~125,can set the warning limitation, the use of seven segments LCD that can be display the current temper ature. The paper focuses on providing a software and hardware system components circuit, in troduced the theory of DS18B20, the founctions and applications of STC89C51. The circuit design reasonable function and practical structure is simple.

Key words：Monolithic integrated circuit STC89C52RC；temperature sensor DS18B20 ；LCD Manifestation

目　　录

引言 1
1　选题的背景和意义 1
1.1 设计任务与要求 1
1.2 设计目的 1
2　总体设计方案 2
2.1 方案阐述 2
2.2 基本思路 2
2.3 方案选择与论证 2
2.4 系统设计原理 4
2.5 系统组成 4
3　硬件设计 4
3.1 硬件原理框图 4
3.2 硬件结构及原理 4
4　软件设计 12
4.1 软件编写工具 12
4.2 总体设计 13
4.3 主程序流程 13
4.4 程序分析 14
5　制板流程 ..... 错误！未定义书签。
6　系统调试 18
6.1 硬件调试 18
6.2 软件调试 19
6.3 调试结果 19
7　结论 21
谢 辞 ..... 错误！未定义书签。
参考文献 21
附　录 22

引言

温度是一个十分重要的物理量，对它的测量与控制有着十分重要的意义。随着现代工农业技术的发展及人们对生活环境要求的提高，人们也迫切需要检测与控制温度。在人类的生活环境中，温度扮演者极其重要的角色。无论你生活在哪里，从事什么工作，无时无刻不在与温度打着交道。自 18 世纪工业革命以来，工业发展对是否能掌握温度有着绝对的联系。在冶金、钢铁、石化、水泥、玻璃、医药等等行业，可以说几乎 80%的工业部门都不得不考虑着温度的因素。

信息科学和微电子技术的飞速发展给控制领域带来了巨大的飞跃；控制技术更加趋向自动化和智能化，为无数的使用者带来了方便。在控制领域里，温度是一个常见的名词，然而它所带来的技术问题和所起的作用却是非同一般的。在控制领域中，对温度的控制有着举足轻重的作用。例如陶瓷的烧烤，只有控制住温度的适度，才能制作出一个完美的艺术品，否则只是一件废品；还有如酿酒的过程，也需要对温度进行控制。可见，在生活的许许多多的方面都有着对温度进行感知和控制的需要。

1　选题的背景和意义

随着社会的发展特别是工业的发展，人民生活的改善，安全问题变得更加重要。目前，在许多情况下，都需要对环境的温度进行限定，其中包括人的生活工作环境、仪器设备的工作环境以及动植物的生长环境等。如果环境温度超过或者低于限制值，必定对所处环境的人或设备造成影响，甚至给个人和社会造成巨大的损失。

因此，在某些特定环境内使用温度报警器来对温度进行实时监控并做到超温报警，而使用单片微型计算机实时控制温度报警系统则是其中的一种重要方式。

主要是了解了单片机微型计算机实时控制的温度测量报警系统的历史与现状，根据现实生活的需要以及已掌握的理论知识，制定出单片微型计算机实时控制的温度报警系统硬件、软件的设计方案，把温度传感器这个单独的器件，配以一些其他电路，让它实现探测温度、显示温度、并且超高温、低温报警，并进行调试验证方案的可行性，最终完成设计。

1.1 设计任务与要求

（1）温度检测：通过传感器和系统连接，能实时准确检测到现场温度。

（2）温度显示：把现场实时检测到的温度值显示出来。

（3）报警温度设定和报警：根据需要可以设置报警温度，并且当达到报警温度时 会发出报警提示。

1.2 设计目的

（1）通过对温度报警电路的设计、安装和调试，掌握温度报警电路的工作原理和运算放大器在实际电子电路中的应用。

（2）学会运用 keil 软件编写程序，提高分析程序的能力。

（3）进一步熟悉 STC89C52RC 单片机外部引脚的线路连接方法。

（4）熟悉 LCD1602 使用方法。

（5）学会使用 Proteus 仿真软件对设计的电路进行仿真，验证电路功能是否正确。

（6）熟悉制作电路板的流程。

2 总体设计方案

2.1 方案阐述

通过分析可以知道，所要设计温度检测与报警装置的温度测量范围为-55~125℃，在 lcd 上并能设置上下限报警温度，超过或低于限制温度发出报警提示。该系统主要由温度测量和数据采集两部分电路组成，实现的方法有很多种，下面将列出两种在日常生活中和工农业生产中经常用到的实现方案。

2.2 基本思路

利用单片机的 P2.5、P2.6、P2.7 作为 LCD1602 的控制位，P0 口作为数据位控制 LCD1602，用 LCD1602 显示被测量温度的数值、上下限报警温度；P3.4、P3.5 、P3.6、P3.7 口控制手动控制按键，用来调节上下限报警温度；P2.0 口作为 18b20 的接入端；P2.口作为继电器的接入端。

2.3. 方案选择与论证

对采集的温度数值正确的显示是程序设计的核心，其测量的结果要求准确度高，测量过程要方便。

方案一：采用热电偶温差电路测温，温度检测部分可以使用低温热偶，热电偶由两个焊接在一起的异金属导线所组成，热电偶产生的热电势由两种金属的接触电势和单一导体的温差电势组成。将两种不同材料的导体或半导体 A 和 B 连接起来,构成一个闭合回路,就构成热电偶.温度 t 端为感温端称为测量端, 温度 t0 端为连接仪表端称为参比端或冷端,当导体 A 和 B 的两个执着点 t 和 t0 之间存在温差时,就在回路中产生电动势 EAB(t,t0), 因而在回路中形成电流,这种现象称为热电效应″.这个电动势称为热电势,热电偶就是利用这一效应来工作的.热电势的大小与 t 和 t0 之差的大小有关.当热电偶的两个热电极材料已知时,由热电偶回路热电势的分布理论知热电偶两端的热电势差可以用下式表示：

EAB(t,t0)＝EAB(t)－EAB(t0)

式中 EAB(t,t0)－热电偶的热电势；

EAB(t)－温度为 t 时工作端的热电势；

EAB(t0)－温度为 t0 时冷端的热电势.

从上式可看出!当工作端的被测介质温度发生变化时,热电势随之发生变化,因此,只要测出 EAB(t,t0)和知道 EAB(t0)就可得到 EAB(t),将热电势送入显示仪表进行指示

或记录,或送入微机进行处理,即可获得测量端温度 t 值。数据采集部分则使用带有 A/D 通道的单片机，在将随被测温度变化的电压或电流采集过来，进行 A/D 转换后，就可以用单片机进行数据的处理，在显示电路上，就可以将被测温度显示出来。热电偶的优点是工作温度范围非常宽，且体积小，但是它们也存在着输出电压小、容易遭受来自导线环路的噪声影响以及漂移较高的缺点，并且这种设计需要用到 A/D 转换电路，感温电路比较麻烦。

系统主要包括对A/D0809 的数据采集，自动手动工作方式检测，温度的显示等，这几项功能的信号通过输入输出电路经单片机处理。此外还有复位电路，晶振电路，启动电路等。故现场输入硬件有手动复位键、A/D 转换芯片，处理芯片为52芯片，执行机构有4 位数码管、报警器等。

方案二：采用数字温度芯片DS18B20 测量温度，输出信号全数字化。便于单片机处理及控制，省去传统的测温方法的很多外围电路。且该芯片的物理化学性很稳定，它能用做工业测温元件，此元件线形较好。在0—100 摄氏度时，最大线形偏差小于1 摄氏度。DS18B20 的最大特点之一采用了单总线的数据传输，由数字温度计DS18B20和微控制器STC89C52RC构成的温度测量装置,它直接输出温度的数字信号,可直接与计算机连接。这样,测温系统的结构就比较简单,体积也不大。采用单片机控制，软件编程的自由度大，

可通过编程实现各种各样的算术算法和逻辑控制，而且体积小，硬件实现简单，安装方便。既可以单独对多DS18B20控制工作，还可以与PC 机通信上传数据，下图为18b20的温度与数据的关系。

温度/℃	二进制表示	十六进制表示
+125	00000111 11010000	07D0H
+25.0625	00000001 10010001	0191H
+10.125	00000000 10100010	00A2H
+0.5	00000000 00001000	0008H
0	00000000 00000000	0000H
-0.5	11111111 11111000	FFF8H
-10.125	11111111 01011110	FF5EH
-25.0625	11111110 01101111	FE6FH
-55	11111100 10010000	FC90H

图1 温度与数据关系图

该系统利用STC89C52RC芯片控制温度传感器DS18B20进行实时温度检测并显示，能够实现快速测量环境温度，并可以根据需要设定上下限报警温度。以此来对某些时间点的温度数据进行存储，利用键盘来进行调时和温度查询。

从以上两种方案，容易看出方案一的测温装置可测温度范围宽、体积小，且复杂，线性误差较大，适合实验。方案二的测温装置电路简单、精确度较高、实现方便、软件设计也比较简单，故本次设计采用了方案二。

2.4 系统设计原理

利用温度传感器 DS18B20 可以直接读取被测温度值，进行转换的特性，模拟温度值经过 DS18B20 处理后转换为数字值，然后送到单片机中进行数据处理，并与设置的温度报警限比较，超过限度后通过扬声器报警。同时处理后的数据送到 LCD 中显示。

2.5 系统组成

本课题以是 STC89C52RC 单片机为核心设计的一种数字温度控制系统，系统整体硬件电路包括，传感器数据采集电路，温度显示电路，上下限报警调整电路，单片机主板电路等组成。

系统框图主要由主控制器 STC89C52RC、单片机复位、报警按键设置、时钟振荡、LCD 显示、温度传感器组成。

3　硬件设计

3.1 硬件原理框图

温度检测与报警系统设计由主控制器 STC89C52RC、单片机复位、报警温度按键设置、时钟振荡、LCD1602 显示、温度传感器及继电器组成。

原理框图如图 2 所示：

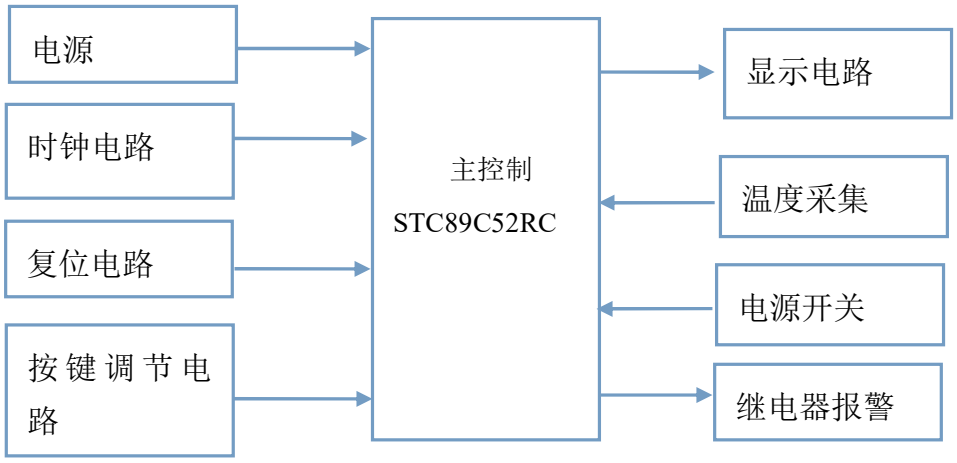


图 2 原理框图

3.2 硬件结构及原理

3.2.1 STC89C52RC 单片机工作电路

本设计的核心是单片机电路，考虑到需要一个中断输入，存储容量、外部接口对单片机端口的需要以及兼顾到节约成本的原则，选用了常用的 STC89C52RC 单片机。STC89C52RC 是低功耗、高性能、经济的 8 位 CMOS 微处理器，工作频率为 0—40MHz，内置 8K 字节可编程只读闪存，512 位的内部 RAM，16 位可编程 I／O 总线。它采用 Atmel 公司的非易储器制造技术，与 MCS-51 的指令设置和芯片引脚可兼容。STC89C52RC 可以按照常规方法进行编程，也可以在线编程。其将通用的微处理器和 Flash 存储器结合在

一起，特别是可反复擦写的 Flash 存储器可有效地降低开发成本。STC89C52RC 工作的最简单的电路是其外围接一个晶振和一个复位电路，给单片机接上电源和地，单片机就可以工作了。

STC89C52RC 各管脚功能如下：

VCC：供电电压。

GND：接地。

P0 口：P0 口为一个 8 位漏级开路双向 I/O 口，每脚可吸收 8TTL 门电流。当 P0 口的管脚第一次写 1 时，被定义为高阻输入。P0 能够用于外部程序数据存储器，它可以被定义为数据/地址的低八位。在 FIASH 编程时，P0 口作为原码输入口，当 FIASH 进行校验时，P0 输出原码，此时 P0 外部必须接上拉电阻。

P1 口：P1 口是一个内部提供上拉电阻的 8 位双向 I/O 口，P1 口缓冲器能接收输出 4TTL 门电流。P1 口管脚写入 1 后，被内部上拉为高，可用作输入，P1 口被外部下拉为低电平时，将输出电流，这是由于内部上拉的缘故。在 FLASH 编程和校验时，P1 口作为低八位地址接收。

P2 口：P2 口为一个内部上拉电阻的 8 位双向 I/O 口，P2 口缓冲器可接收，输出 4 个 TTL 门电流，当 P2 口被写“1”时，其管脚被内部上拉电阻拉高，且作为输入。并因此作为输入时，P2 口的管脚被外部拉低，将输出电流。这是由于内部上拉的缘故。P2 口当用于外部程序存储器或 16 位地址外部数据存储器进行存取时，P2 口输出地址的高八位。在给出地址“1”时，它利用内部上拉优势，当对外部八位地址数据存储器进行读写时，P2 口输出其特殊功能寄存器的内容。P2 口在 FLASH 编程和校验时接收高八位地址信号和控制信号。

P3 口：P3 口管脚是 8 个带内部上拉电阻的双向 I/O 口，可接收输出 4 个 TTL 门电流。当 P3 口写入“1”后，它们被内部上拉为高电平，并用作输入。作为输入，由于外部下拉为低电平，P3 口将输出电流（ILL）这是由于上拉的缘故。

P3口也可作为STC89C52RC的一些特殊功能口，如下表所示。在flash编程和校验时，P3口也接收一些控制信号。

表 3.2.1 P3 口第二功能

引脚	第二功能
P3.0	RXD（串行输入）
P3.1	TXD（串行输出）
P3.2	$\overline{\text{INT0}}$ (外部中断0)
P3.3	$\overline{\text{INT1}}$ (外部中断1)

P3.4	T_0（定时器0外部输入）
P3.5	T_1（定时器1外部输入）
P3.6	$\overline{WR}$ （外部数据存储器写选通）
P3.7	$\overline{RD}$ （外部数据存储器写选通）

RST：复位输入。当振荡器复位器件时，要保持 RST 脚两个机器周期的高电平时间。

ALE/PROG：当访问外部存储器时，地址锁存允许的输出电平用于锁存地址的低位字节。在 FLASH 编程期间，此引脚用于输入编程脉冲。在平时，ALE 端以不变的频率周期输出正脉冲信号，此频率为振荡器频率的 1/6。因此它可用作对外部输出的脉冲或用于定时目的。然而要注意的是：每当用作外部数据存储器时，将跳过一个 ALE 脉冲。如想禁止 ALE 的输出可在 SFR8EH 地址上置 0。此时， ALE 只有在执行 MOVX，MOVC 指令是 ALE 才起作用。另外，该引脚被略微拉高。如果微处理器在外部执行状态 ALE 禁止，置位无效。

/PSEN：外部程序存储器的选通信号。在由外部程序存储器取指期间，每个机器周期两次/PSEN 有效。但在访问外部数据存储器时，这两次有效的/PSEN 信号将不出现。

/EA/VPP：当/EA 保持低电平时，则在此期间外部程序存储器（0000H-FFFFH），不管是否有内部程序存储器。注意加密方式 1 时，/EA 将内部锁定为 RESET；当/EA 端保持高电平时，此间内部程序存储器。在 FLASH 编程期间，此引脚也用于施加 12V 编程电源（VPP）。

XTAL1：反向振荡放大器的输入及内部时钟工作电路的输入。

XTAL2：来自反向振荡器的输出。

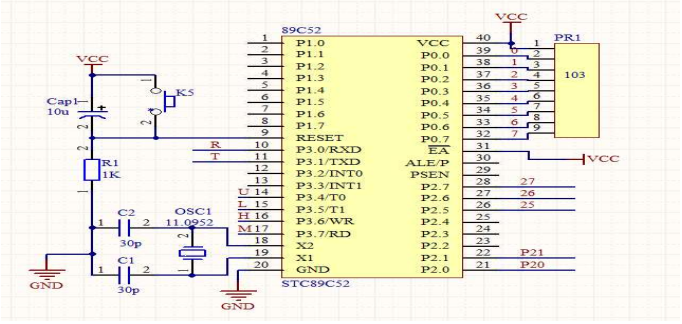


图 3 STC89C52RC 工作电路

3.2.2 STC89C52RC 复位电路

在上电或复位过程中，控制 CPU 的复位状态：这段时间内让 CPU 保持复位状态，而不是一上电或刚复位完毕就工作，防止 CPU 发出错误的指令、执行错误操作，也可以提

高电磁兼容性能。无图 3 STC89C52RC 工作电路论用户使用哪种类型的单片机,总要涉及到单片机复位电路的设计。

而单片机复位电路设计的好坏,直接影响到整个系统工作的可靠性。许多用户在设计完单片机系统,并在实验室调试成功后,在现场却出现了“死机”、“程序走飞”等现象,这主要是单片机的复位电路设计不可靠引起的。

单片机在启动时都需要复位，以使 CPU 及系统各部件处于确定的初始状态，并从初态开始工作。89 系列单片机的复位信号是从 RST 引脚输入到芯片内的施密特触发器中的。当系统处于正常工作状态时，且振荡器稳定后，如果 RST 引脚上有一个高电平并维持 2 个机器周期(24 个振荡周期)以上，则 CPU 就可以响应并将系统复位。

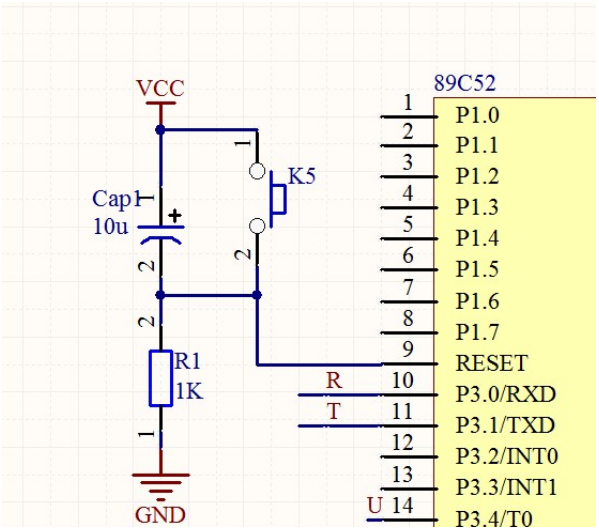


图 4 复位电路

复位电路通常采用上电自动复位和按钮复位两种方式。上电自动复位是通过外部复位电路的电容充电来实现的。只要 VCC 的上升时间不超过 1ms，就可以实现自动上电复位。除了上电复位外，有时还需要按键手动复位。按键手动复位有电平方式和脉冲方式两种。其中电平复位是通过 RST 端经电阻与电源 VCC 接通而实现的。兼有上电复位和按钮复位的电路。在单片机设计中，若有外部扩展的 I/O 接口电路需初始复位，如果它们的复位端和 MCS-51 单片机的复位端相连，复位电路中的 R、C 参数要受到影响，这时复位电路中的 R、C 参数要统一考虑以保证可靠的复位。如果单片机 MCS-51 单片机与外围 I/O 接口电路的复位电路和复位时间不完全一致，单片机初始化程序不能正常运行，外围/O 接口电路的复位也可以不和 MCS-51 单片机复位端相连，仅采用独立的上电复位电路。一般来说，单片机的复位速度比外围 I/O 快些。若 RC 上电复位电路接 MCS-51 单片机和外围电路复位端，则能使系统可靠地同步复位。为保证系统可靠复位，在初始化程序中应用到一定的复位延迟时间。复位电路软件程序跑飞或者硬件发生错误的时候产生一个复位信号，控制 MCS-51 单片机从 0000H 单元开始执行程序，重新执行软件程序。此电路的输出端 RESET 接在单片机的复位引脚。

3.2.3 STC89C52RC 时钟电路

时钟在单片机中非常重要，单片机各功能部件的运行都是以时钟频率为基准。时钟频率直接影响单片机的速度，时钟电路的质量也直接影响单片机系统的稳定性。常用的时钟电路有两种方式，一种是内部时钟方式，另一种为外部时钟方式。内部时钟方式电路图如图 5 所示：

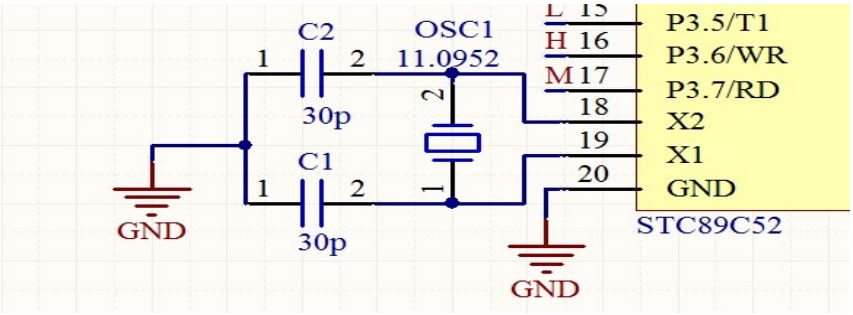


图 5 时钟电路

MCS-51 单片机内部有一个用与构成振荡器的高增益反相放大器，该高增益反相放大器的输入端为芯片引脚 XTAL1，输出端为引脚 XTAL2。这两个引脚接石英晶体振荡器和微调电容，就构成一个稳定的自激振荡器电路。电路中的电容 C1 和 C2 典型值通常选择为 20PF 左右。对外接电容的值虽然没有严格的要求，但是电容的大小会影响振荡器频率的高低、振荡器的稳定性和起振的快速性。晶体的振荡频率的范围通常是在 1.2MHz—12MHz 之间。晶体的频率越高，则系统的时钟频率也就越高，单片机的运行速度也就越快。为了提高温度稳定性，应采用温度稳定性能好的 NPO 高频电容。MCS-51 单片机常选择振荡频率 12MHz 的石英晶体。外部时钟方式是使用外部振荡脉冲信号，常用于多片 MCS-51 单片机同时工作，以便于同步。对外部脉冲信号只要求高电平的持续时间大于 20us，一般为低于 12MHz 的方波。外部的时钟源直接接到 XTAL2 端，直接输入到片内的时钟发生器上。由于 XTAL2 的逻辑电平不是 TTL 的，因此要外接一个 4.7k～10k 的上拉电阻。这次的设计采用 MCS-51 的内部时钟方式。因为外部时钟方式是用外部振荡脉冲信号，用于多片 MCS-51 单片机同时工作。在这次设计中只用一个 MCS-51 单片机，不需要振荡脉冲信号。

3.2.4 STC89C52RC 按键电路

按键是实现人机对话的比较直观的接口，可以通过按键实现人们想让单片机做的不同的工作。常用的键盘接口分为独立式键盘接口和矩阵式键盘接口。独立式键盘接口就是各键相互独立，每个按键各接一根输入线，一根输入线上的按键工作状态不影响其他输入线的工作状态。通过检测输入电平状态可以很容易判断哪个按键被按下了。独立式按键电路配置灵活，软件实现简单。但每个按键需占用一根输入口线，在按键数量较多时，需要较多的输入口线而且电路结构繁杂，故此种键盘适用于按键较少或操作速度较高的场合。而矩阵式键盘有行线和列线组成，按键位于行，列的交叉点上，适用于按键

较多的场合。下图就是一种比较典型的独立按键电路，在按键没有按下的时候，I/O 口采集到的是高电平，当按键按下去的时候，I/O 口采集到的是低电平。

3.2.5 STC89C52RC 显示电路

1602 字符型 LCD 模块的应用非常广泛,而各种液晶厂家均有提供几乎都是同样规格的 1602 模块或兼容模块,尽管各厂家的对其各自的产品命名不尽相同;1602 字符型 LCD 模块最初采用的LCD控制器采用的是HD44780，在各厂家生产的1602模块当中，基本上也都采用了与之兼容的控制IC，所以从特性上基本上是一样的；当然，很多厂商提供了不同的字符颜色、背光色之类的显示模块。

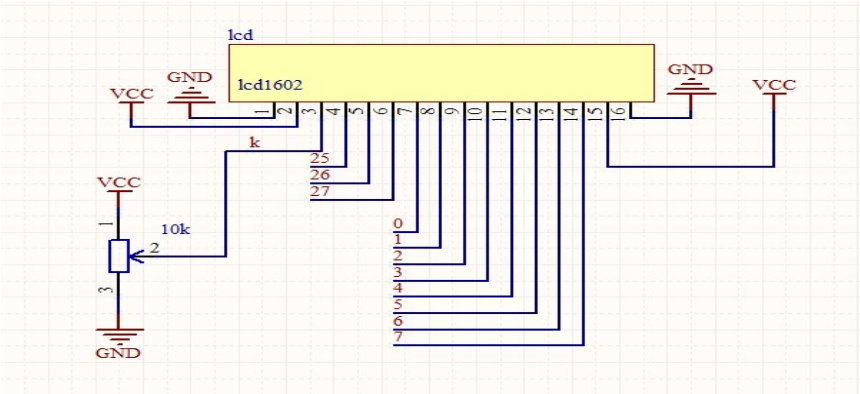


图6 液晶显示电路

表3.2.5 LCD管脚功能

编号	符号	引脚说明	编号	符号	引脚说明
1	VSS	电源地	9	D2	Data I/O
2	VDD	电源正极	10	D3	Data I/O
3	VL	液晶显示偏压信号	11	D4	Data I/O
4	RS	数据/命令选择端（H/L）	12	D5	Data I/O
5	R/W	读/写端(H/L)	13	D6	Data I/O
6	E	使能信号	14	D7	Data I/O
7	D0	Data I/O	15	BLA	电源背光正极
8	D1	Data I/O	16	BLK	电源背光负极

3.2.6 DS18B20

DS18B20 的读写时序和测温原理与 DS1820 相同,只是得到的温度值的位数因分辨率不同而不同，且温度转换时的延时时间由 2s 减为 750ms。 DS18B20 测温原理：低温度系数晶振的振荡频率受温度影响很小，用于产生固定频率的脉冲信号送给计数器 1。高温度系数晶振 随温度变化其振荡率明显改变，所产生的信号作为计数器 2 的脉冲输入。计数器 1 和温度寄存器被预置在－55℃所对应的一个基数值。计数器 1 对 低温度系数

晶振产生的脉冲信号进行减法计数，当计数器 1 的预置值减到 0 时，温度寄存器的值将加 1，计数器 1 的预置将重新被装入，计数器 1 重 新开始对低温度系数晶振产生的脉冲信号进行计数，如此循环直到计数器 2 计数到 0 时，停止温度寄存器值的累加，此时温度寄存器中的数值即为所测温度。



图 7 DS18B20

DS18B20 功能特点：

（1）采用单总线技术，与单片机通信只需要一根 I/O 线，在一根线上可以挂接多个 DS18B20。

（2）每只 DS18B20具有一个独有的，不可修改的64位序列号，根据序列号访问地应的器件。

（3）低压供电，电源范围从3~5V，可以本地供电，也可以直接从数据线上窃取电源（寄生电源方式）。

（4）测温范围为-55℃~+125℃,在-10℃~85℃范围内误差为±0.5℃。

（5）可编辑数据为9~12位，转换12位温度时间为750ms（最大）。

（6）用户可自设定报警上下限温度。

（7）报警搜索命令可识别和寻址哪个器件的温度超出预定值。

（8） DS18B20的分辩率由用户通过 EEPROM 设置为9~12位。

（9） DS18B20 可将检测到温度值直接转化为数字量，并通过串行通信的方式与主控制器进行数据通信。

DS18B20 有 4 个主要的数据部件：

（1） 光刻 ROM 中的 64 位序列号是出厂前被光刻好的，它可以看作是该 DS18B20 的地址序列码。64 位光刻 ROM 的排列是：开始 8 位 （28H）是产品类型标号，接着的 48 位是该 DS18B20 自身的序列号，最后 8 位是前面 56 位的循环冗余校验码（CRC=X8+X5+X4+1）。光刻 ROM 的作用 是使每一个 DS18B20 都各不相同，这样就可以实现一根总线上挂接多个 DS18B20 的目的。

（2） DS18B20 中的温度传感器可完成对温度的测量，以 12 位转化为例：用 16 位符号扩展的二进制补码读数形式提供，以 0.0625℃/LSB 形式表达，其中 S 为符号位。

（3） DS18B20 温度传感器的存储器 DS18B20 温度传感器的内部存储器包括一个高速暂存 RAM 和一个非易失性的可电擦除的 EEPRAM,后者存放高温度和低温度触发器 TH、TL 和结构寄存器。

（4） 配置寄存器。

DS18B20 内部结构及功能：

DS18B20的内部结构主要包括：寄生电源，温度传感器，64位 ROM 和单总线接口，存放中间数据的高速暂存器 RAM，用于存储用户设定温度上下限值的 TH 和 TL 触发器，存储与控制逻辑，8位循环冗余校验码（CRC）发生器等7部分。

温度的读取：

DS18B20 在出厂时以配置为 12 位，读取温度时共读取 16 位，所以把后 11 位的 2 进制转化为 10 进制后在乘以 0.0625 便为所测的温度，还需要判断正负。前 5 个数字为符号位，当前 5 位为 1 时，读取的温度为负数；当前 5 位为 0 时，读取的温度为正数。

DS18B20 的初始化：

（1）先将数据线置高电平“1”。

（2）延时（该时间要求的不是很严格，但是尽可能的短一点）。

（3）数据线拉到低电平“0”。

（4）延时 750 微秒（该时间的时间范围可以从 480 到 960 微秒）。

（5）数据线拉到高电平“1”。

（6）延时等待（如果初始化成功则在 15 到 60 毫秒时间之内产生一个由 DS18B20 所返回的低电平“0”。据该状态可以来确定它的存在，但是应注意不能无限的进行等待，不然会使程序进入死循环，所以要进行超时控制

（7）若 CPU 读到了数据线上的低电平“0”后，还要做延时，其延时的时间从发出的高电平算起（第（5）步的时间算起）最少要 480 微秒。

（8）将数据线再次拉高到高电平“1”后结束。

DS18B20 的写操作：

（1）数据线先置低电平“0”。

（2） 延时确定的时间为 15 微秒。

（3） 按从低位到高位的顺序发送字节（一次只发送一位）。

（4）延时时间为 45 微秒。

（5）将数据线拉到高电平。

（6） 重复上（1）到（6）的操作直到所有的字节全部发送完为止。

（7） 最后将数据线拉高。

DS18B20 的读操作：

（1）将数据线拉高“1”。

（2）延时 2 微秒。

（3）将数据线拉低“0”。

（4）延时 15 微秒。

（5）将数据线拉高“1”。　　（6）延时 15 微秒。

（7）延时 30 微秒。

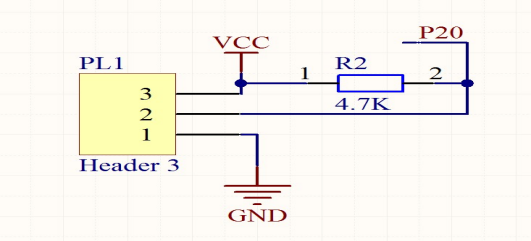


图 8　DS18B20 电路

3.2.7 继电器

继电器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、打印机、复印机、报警器、电子玩具、汽车电子设备、电话机、定时器等电子产品中作发声器件。 继电器主要分为压电式继电器和电磁式继电器两种类型。继电器在电路中用字母“H”或“HA”（旧标准用“FM”、“LB”、“JD”等）表示。



图 9 继电器

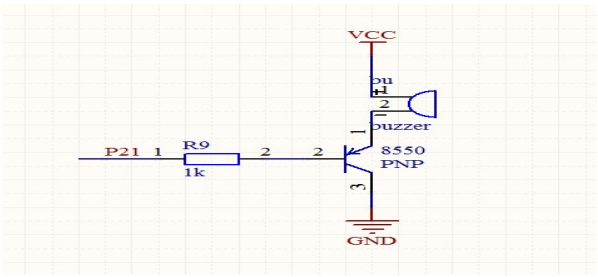


图 10　继电器电路图

4　软件设计

本设计智能温度报警系统由温度采集、信号处理、温度监测、输出控制四部分组成。它通过预先设在单片机中的高低温度值来对非常温度值进行报警，也可以通过安检的控制对温度进行高低的设定最低可以由 0℃开始进行温度的设定，从 DSI8B20 采集到的温度经信号调理电路处理后直接送入单片机进行刷新。微控制器根据信号数据及设定的各种控制参数，按照嵌入的软件控制规律执行计算与处理，自动显示温度值、输出相应的控制信号，并根据当前状态输出正常、报警等信号，同时将各种数据通过数码管进行显示监控。

4.1 软件编写工具

软件编写工具选用美国 Keil Software 公司出品的 51 系列兼容单片机 C 语言软件开发系统 Keil C51，与汇编相比，C 语言在功能上、结构性、可读性、可维护性上有明显的优势。

4.2 总体设计

该温度检测的软件设计采用 C51 语言编程。系统程序由主程序、温度设定子程序、温度采集子程序、温度刷新子程序、液晶显示程序和按键子程序等部分组成(程序流程图如图 12 所示)。

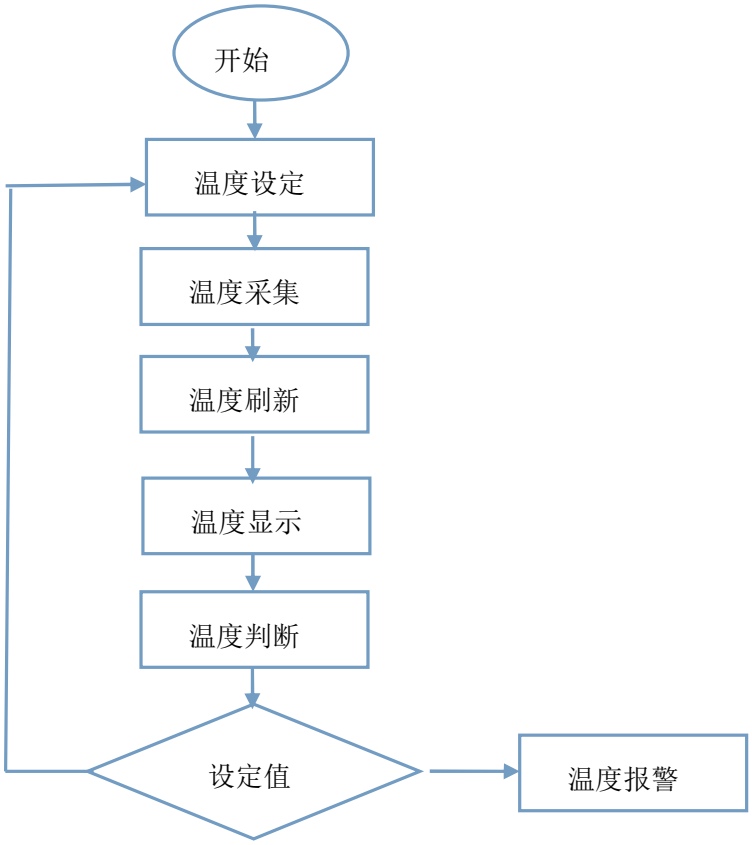


图 11　系统的软件设计流程

4.3 主程序流程

主程序的主要功能是负责温度的实时显示、读出并处理 DS18B20 的测量的当前温度值，其程序流程见图 13 所示:

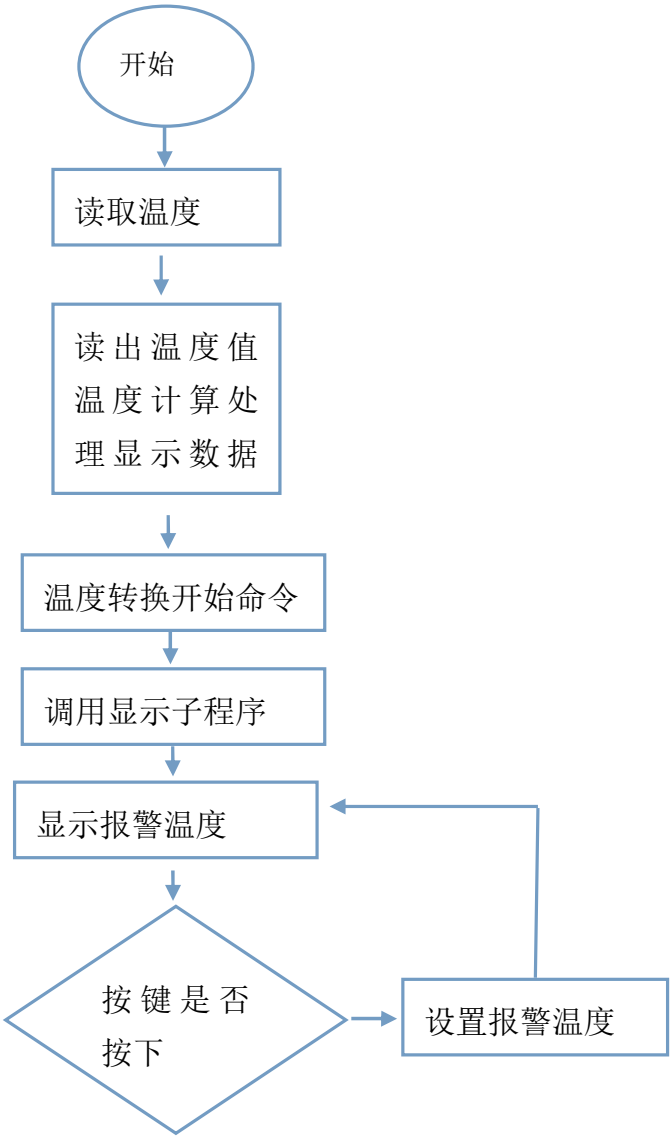


图 12　主程序流程

4.4　程序分析

4.4.1 读出温度子程序

读出温度子程序的主要功能是读出 RAM 中的 9 字节，在读出时需进行 CRC 校验，校验有错时不进行温度数据的改写。

程序如下：

```
void ReadTemperature(void)
{
    unsigned char L=0;
    unsigned char H=0;
    Init_DS18B20();
    WriteOneChar(0xCC);//跳过读序号列号的操作
    WriteOneChar(0x44);//启动温度转换
    delay_18B20(100);
```

```
    Init_DS18B20();
    WriteOneChar(0xCC);//跳过读序号列号的操作
    WriteOneChar(0xBE);//读取温度寄存器等（共可读 9 个寄存器前两个就是 温度
    L=ReadOneChar();
    H=ReadOneChar();
    if(H>0xf8)              //高位前五位为 1 时温度是负
    {
        L=~L+1;
        H=~H;
        f=0;                                    //读取温度为负时 fg=0
        sdata = L/16+H*16;              //整数部分
        xiaoshu1 = (L&0x0f)*10/16;              //小数第一位
        xiaoshu2 = (L&0x0f)*100/16%10;//小数第二位
    }
//传感器返回值除 16 得实际温度值
//为了得到 2 位小数位，先乘 100，再除 16，考虑整型数据长度，
    else
    {
        f=1;
        sdata = L/16+H*16;                  //整数部分
        xiaoshu1 = (L&0x0f)*10/16;              //小数第一位
        xiaoshu2 = (L&0x0f)*100/16%10;//小数第二位
    }
}
```

4.4.2 显示数据刷新子程序

显示数据刷新子程序主要是对分离后的温度显示数据进行刷新操作，当标志位位为 1 时将符号显示。

程序如下：

```
void display(unsigned int date)      //显示温度，
{
    if(date/100==0) //100 度内
    {
        write_com(0x84);
        write_data(shuzu[date/10]);//十位
        write_com(0x85);
```

```
            write_data(shuzu[date%10]);//个位
            write_com(0x86);
            write_data(shuzu[13]);//小数点
            write_com(0x87);
            write_data(shuzu[xiaoshu1]);//小数点后一位
            write_com(0x88);
            write_data(shuzu[xiaoshu2]); //小数点后二位
            write_com(0x89);
            write_data(shuzu[12]);
        }
        else if(date/100==1)      //100 度上
        {
            write_com(0x84);
            write_data(shuzu[date/100]);//百位
            write_com(0x85);
            write_data(shuzu[(date-100)/10]);//十位
            write_com(0x86);
            write_data(shuzu[date%10]);//个位
            write_com(0x87);
            write_data(shuzu[13]);//点
            write_com(0x88);
            write_data(shuzu[xiaoshu1]);//小数点后一位
            write_com(0x89);
            write_data(shuzu[xiaoshu2]);//小数点后二位
        }
    }
```

4.4.3 按键扫描子程序

第一步：平时没有按键被触发时，不会执行程序。

第二步：一旦有按键被按下，执行去抖动延时，再判定。

第三步：如果按键按下的时间超过了去抖动延时，则触发按键，马上把按键值锁定，防止按住按键不松手后一直触发。

第四步：等按键松开后，自锁不执行，为下一次自锁做准备。

第五步：以上整个过程，就是识别按键 IO 口下降沿触发的过程。

程序如下：

```
void key()          //按键加减标尺
```

```
{
   if(keym==0)//判定
   {
     delay_50us(100);//延时
     if(keym==0)
     {
       y++;//转换加减的数值
       if(y==2)m=1;
       else if(y==3)m=5;//数值 5
      else if(y==4)m=10; //数值 10
       else if(y>=5)y=1; //数值 1
      while(keym==0);//自锁
     }
   }
   if(keyh==0)//上下限转换按键
   {
     delay_50us(100);
     if(keyh==0)
     {
       o++;
          if(o>=3)o=0;//1 为上限选中，2 为下限选中，0 为隐藏
      while(keyh==0);
     }
   }
/////调节上限温度
   if(keyu==0&o==1)//上限加按键
   {
     delay_50us(100);
     if(keyu==0&o==1)
     {
        u=u+m; while(keyu==0);
     }
   }
   if(keyl==0&o==1) //上限减按键
```

```
    {
       delay_50us(100);
       if(keyl==0&o==1)
       {
         u=u-m; while(keyl==0);
       }
    }
//// 调节下限温度
    if(keyu==0&o==2)//下限加按键
    {
       delay_50us(100);
       if(keyu==0&o==2)
       {
          l=l+m;while(keyu==0);
       }
    }
    if(keyl==0&o==2)//下限减按键
    {
       delay_50us(100);
       if(keyl==0&o==2)
       {
         l=l-m;while(keyl==0);
       }
    }
}
```

6 系统调试

6.1 硬件调试

（1）目测，检查外部的各种元件或者电路是否有断点。

（2）温度检测与报警装置在没通电的情况下，用数字万用表的二极管档检查电路板线路是否导通或者短路。

（3）加电检测，给电路板加电检测所有的插座或者硬件的电源是否符合要求的值。

6.2 软件调试

6.2.1 仿真软件

随着科技的发展，“计算机仿真技术”已成为许多设计部门重要的前期设计手段。它具有设计灵活，仿真的结果与测试结果较为接近。Proteus 软件是英国 L 阿宝 center electronics 公司出版的 EDA 工具软件。它不仅具有其他 EDA 工具软件的仿真功能，还能仿真单片机及外围器件。它是目前最好的仿真单片机及外围器件的工具。该软件功能强大。先在 proteus 上按照电路图连接好电路图，打开程序 HEX 文件进行仿真。仿真结果如下，lcd 能正常显示温度和上下限温度，按键可以准确调节上下限温度，仿真结果与理论接近。

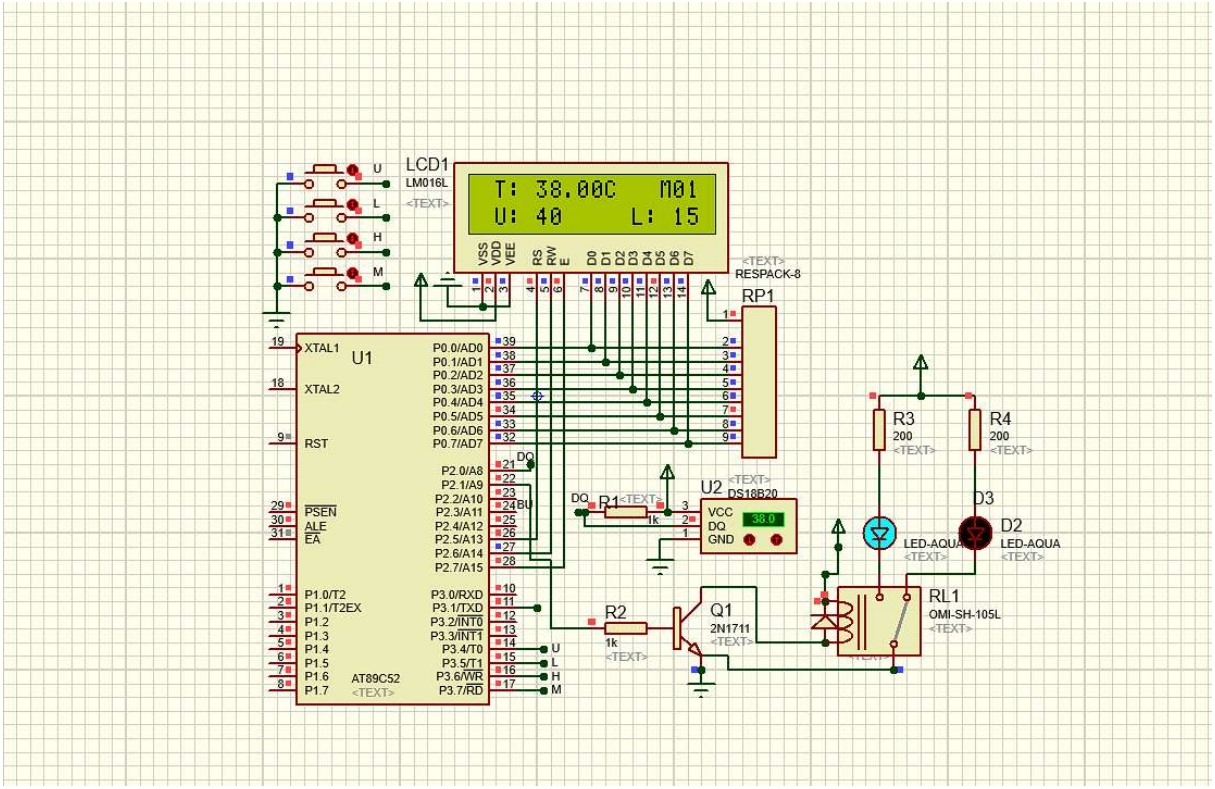


图13 仿真图

6.3 调试结果

开启电源，打开开关，LCD1206 显示欢迎并开始测温。当温度超过上限温度或低于下限温度时报警提示正常，以下是实时温度的一些测量值，因 1602 的显示屏大小不能设置到小数部分的报警温度，只能设定到整数部分的报警温度。

表 6.3.1 温度测试结果

温度测量							
温度计温度	10.3℃	23.6℃	31.2℃	42.1℃	54.3℃	62.1℃	73.8℃
测量值	10.2℃	23.3℃	31.0℃	41.8℃	54.8℃	61.5℃	74.1℃

相对误差	0.9%	0.8%	0.6%	0.7%	0.9%	0.9%	0.4%
上限报警温度测量							
上限设定报警温度	50℃	55℃	60℃	65℃	70℃	75℃	80℃
报警温度	>=50.0℃	>=55.0℃	>=60.0℃	>=65.0℃	>=70.0℃	>=75.0℃	>=80.0℃
下限报警温度测量							
下限设定报警温度	10	15	20	25	30	35	40
报警温度	<=10.0℃	<=15.0℃	<=20.0℃	<=25.0℃	<=30.0℃	<=35.0℃	<=40.0℃

7 结论

该智能温度报警系统结构简单，具有一定的实际应用价值。该智能温度报警系统只是 DS18B20 在温度控制领域的一个简单实例，此外，还能广泛地应用于各种工业生产领域，如建筑，仓储等行业，特别是在蔬菜大棚的应用上，本次设计虽然完成了，但是次设计也还是有些许不足的地方，与实际温度计测量到的温度还是有差距，因为能力的问题，该智能温度报警系统精确度最高可达到相差 0.1℃，最大误差为 1.5℃；能够设定的上限温度范围为 0℃到 120℃，下限温度为-50℃到 99℃。

参考文献

[1] 康华光，陈大钦. 电子技术基础模拟部分［M］.北京：高等教育出版社，1999：1－495

[2] 康华光. 电子技术基础数字部分［M］.北京：高等教育出版社，2004

[3] 曹巧媛. 单片机原理及应用［M］.北京：电子工业出版社，2002：1－351

[4] 沈文，詹卫前. 单片机C语言开发入门指导［M］.北京：清华大学出版社，2003：17－131

[5] 黄智伟. 全国大学生电子设计竞赛系统设计［M］.北京：北京航空航天大学出版社，2006

[6] 林捷，杨绪业. 模拟电路与数字电路［M］.北京：人民邮电出版社，2007

[7] 阮忠，林金宝. 综合电子电路应用[M].北京：机械工业出版社，2004：147-338

[8] 耿德根，宋建国. AVR 高速嵌入式单片机原理与应用［M］.北京：北京航空航天大学出版社，05：358－410

[9] 周坚.单片机 C 语言轻松入门［M］. 北京：北京航空航天大学出版社，2006.

[10] 曹天汉.单片机原理与接口技术［M］. 北京：电子工业出版社，2006.

[11] 张秀国.单片机 C 语言程序设计教程与实训［M］. 北京：北京大学出版社，2008.

[12] 王莲英.模拟电子技术［M］. 北京：高等教育出版社，2008.

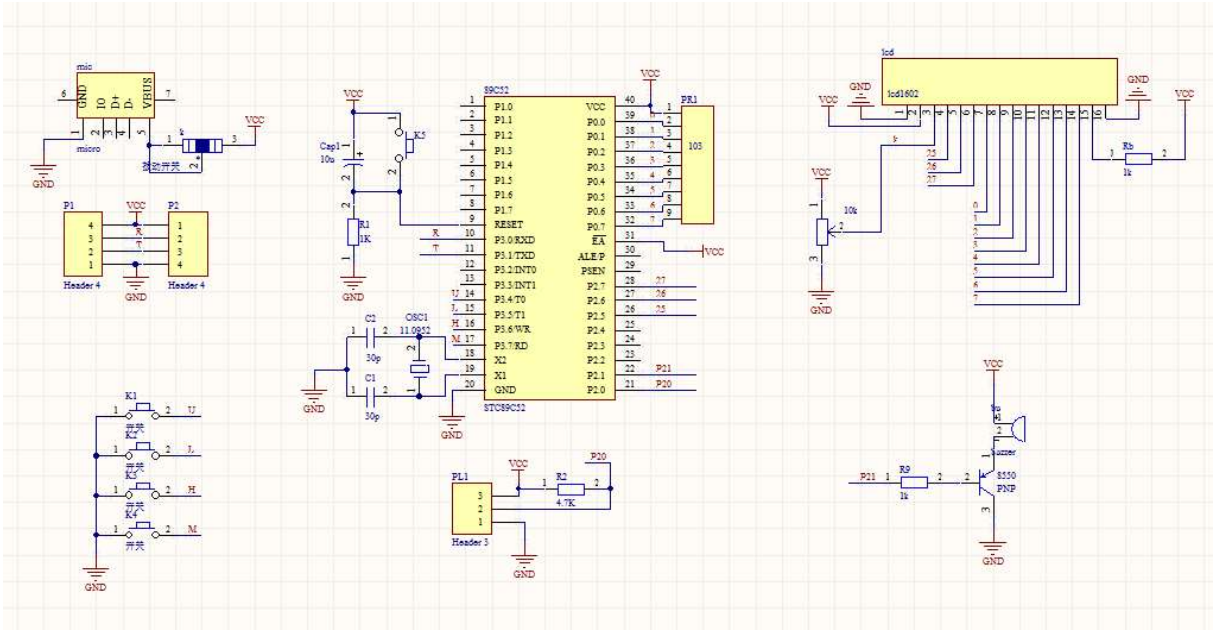
[13] 赵文博，刘文涛.单片机语言 C51 程序设计［M］. 北京：人民邮电出版社，2005.

[14] R.C.Gonzalaz,P.Wintz,Digital image processing,Addison-Wesley Publishing cmp [R].1997

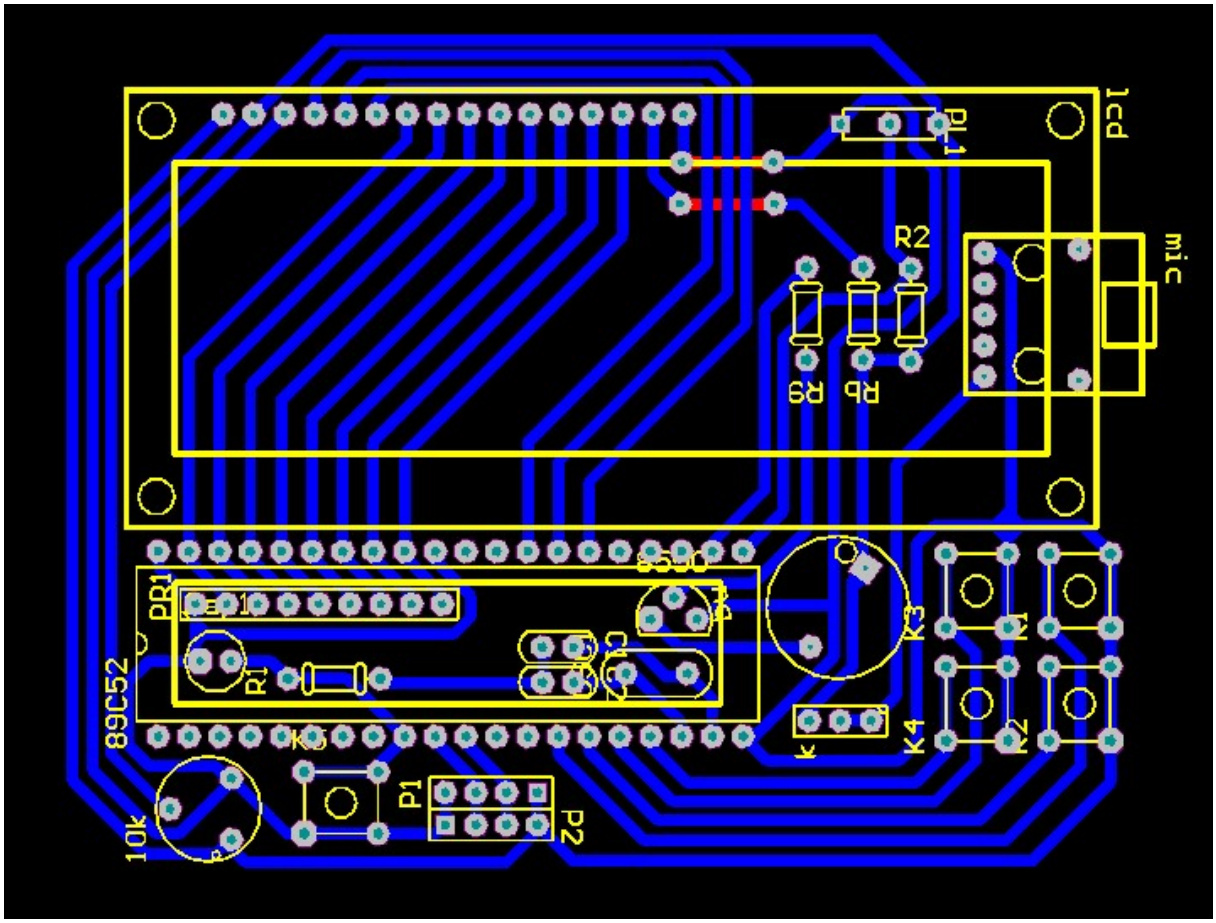
[15] Power Integration. Inc.National semiconductor datasheets[R].2005

附 录

原理图


mic
micro
GND
VCC
P1
P2
Header 4
Header 4
K1
K2
K3
K4
K5
Cap1
10u
R1
1K
89C52
STC89C52
PR1
103
C2
C1
30p
30p
OSC1
11.0592
PL1
Header 3
R2
4.7K
P20
lcd
lcd1602
10k
Rb
1k
Buzzer
R9
1k
8550
PNP
P21
P20

PCB



C 程序

#include <REGX52.H>

```
sbit keyu=P3^4;//加按键
sbit keyl=P3^5;//减按键
sbit keyh=P3^6;//调节尺度
sbit keym=P3^7; //转换模式
sbit RS=P2^5;//数据/命令选择端（H/L）
sbit RW=P2^6;//读/写端(H/L)
sbit EN=P2^7;//使能
sbit DQ=P2^0;   //DS18B20 接入口
sbit buzz=P2^1;//继电器接口       -
bit f=1;   //正负温度标志位
unsigned char    m=1,y=0,o=0;//寄存器
unsigned int sdata;//存放整数温度
unsigned char xiaoshu1; //存放小数后第一位温度数值
unsigned char xiaoshu2;   //存放小数后第二位温度数值
unsigned char code huany[]=" Welcome to use "; //欢迎使用
unsigned char code tt[]    =" ter thermometer"; //温度计
unsigned char code duqu[]="      loading...      "; //读取
unsigned char code tab1[]=" T:           C     M    ";      //框架
unsigned char code tab2[]="U:              L:       ";
unsigned char code shuzu[]="0123456789+- .*"; //数字,,10 加号，，，11 减号，，，12 空格，，，，13
点，，，，14*
unsigned char u=40;//上限
char l=15,p;
                        //下限
void delay_50us(unsigned int t)      //延时函数
{
 unsigned int j,k;
 for (;t>0;t--)
 {
  for(j=10;j>0;j--)
  {
   for(k=1;k>0;k--);
  }
 }
}
```

```
void button() //报警继电器
{
   unsigned char k=2;
   while(k--)
  {
   buzz=0;delay_50us(200);
   buzz=1;delay_50us(200);
  }
}
///lcd
void write_com(unsigned com)//写操纵
{
     EN=0;
     RS=0;
     RW=0;
     P0=com;
     delay_50us(1);
     EN=1;
     delay_50us(5);
     EN=0;
}
void write_data(unsigned char dat)//写数据
{
     EN=0;
     RS=1;
     RW=0;
     P0=dat;
     delay_50us(1);
     EN=1;
     delay_50us(5);
     EN=0;
}
void init(void)//初始化
{
         delay_50us(5);
```

```
        write_com(0x38);
        delay_50us(1);
        write_com(0x38);
        delay_50us(1);
        write_com(0x38);
        write_com(0x38);
        write_com(0x08);
        write_com(0x01);
        write_com(0x06);
        write_com(0x0C);
}
void delay_18B20(unsigned int i)  //18b20 全局延时
{
   while(i--);
}
//////////////////////////////////////18b20/////////////////////////////////////////////////
//初始化传感器函数
   Init_DS18B20(void)
   {
   unsigned char x=0;
   DQ=1;//DQ 复位
   delay_18B20(10);//稍做延时
   DQ=0;//单片机将 DQ 拉低
   delay_18B20(80);//精确延时大于 480us
   DQ=1;//拉高总线
   delay_18B20(20);
   x=DQ;//稍做延时后如果 x=0 则初始化成功 x=1 则初始化失败
   delay_18B20(30);
   return x;
}
//读一个字节
ReadOneChar(void)
{
     unsigned char i=0;
     unsigned char dat=0;
```

```
    for(i=8;i>0;i--)
    {
        DQ=0;//给脉冲信号
        dat>>=1;
        DQ=1;//给脉冲信号
        if(DQ)
        dat|=0x80;
        delay_18B20(4);
    }
    return(dat);
}
//写一个字节
WriteOneChar(unsigned char dat)
{
    unsigned char i=0;
    for(i=8;i>0;i--)
    {
        DQ=0;
        DQ=dat&0x01;
        delay_18B20(5);
        DQ=1;
        dat>>=1;
    }
    return(dat);
}
void ReadTemperature(void)
{
    unsigned char L=0;
    unsigned char H=0;
    Init_DS18B20();
    WriteOneChar(0xCC);//跳过读序号列号的操作
    WriteOneChar(0x44);//启动温度转换
    delay_18B20(100);
    Init_DS18B20();
    WriteOneChar(0xCC);//跳过读序号列号的操作
```

```
    WriteOneChar(0xBE);//读取温度寄存器等（共可读 9 个寄存器前两个就是 温度
    L=ReadOneChar();
    H=ReadOneChar();
    if(H>0xf8)              //高位前五位为 1 时温度是负
    {
        L=~L+1;
        H=~H;
        f=0;                                    //读取温度为负时 fg=0
        sdata = L/16+H*16;              //整数部分
        xiaoshu1 = (L&0x0f)*10/16;              //小数第一位
        xiaoshu2 = (L&0x0f)*100/16%10;//小数第二位
    }
//传感器返回值除 16 得实际温度值
//为了得到 2 位小数位，先乘 100，再除 16，考虑整型数据长度，
    else
    {
        f=1;
        sdata = L/16+H*16;                  //整数部分
        xiaoshu1 = (L&0x0f)*10/16;              //小数第一位
        xiaoshu2 = (L&0x0f)*100/16%10;//小数第二位
    }
}
//////////////////18b20 结束//////////////
void display0()//显示 welcome     to use the
{
        unsigned char i,w,h=0x8f,i1,w1,h1=0xcf,w2,i2,h2,g=2;
        init();
        for(w=0;w<16;w++)   //显示欢迎
        {
            write_com(h--);//第一行自减左移
            for(i=0;i<16;i++)
            {
                write_data(huany[i]);
                delay_50us(15);
            }
```

```
}
for(w1=0;w1<16;w1++)//第二行自减左移
{
    write_com(h1--);
    for(i1=0;i1<16;i1++)
    {
        write_data(tt[i1]);
        delay_50us(15);
    }
}
delay_50us(5000);//显示 loading
init();
write_com(0x80);
for(w2=0;w2<16;w2++)//向右显示
{
    write_data(duqu[w2]);
    delay_50us(200);
}
while(g--)//小数点循环亮灭
{
    write_com(0x8a);
    for(i2=0;i2<3;i2++)
    {
        write_data(duqu[i2]);

    }
    write_com(0x8a);
    for(h2=10;h2<14;h2++)
    {
        delay_50us(300);
        write_data(duqu[h2]);
        delay_50us(300);
    }
}
```

```
}
void display1()       //显示框架
{
        unsigned char p,q;
        init();//全局清零
        write_com(0x80);
        for(q=0;q<16;q++)//显示 TMHL
        {
            write_data(tab1[q]);
        }
        write_com(0xc1);
        for(p=0;p<16;p++)
        write_data(tab2[p]);
}
void display2()//显示上下限温度 ,尺度
{
    //尺度

        write_com(0x8e);
        write_data(shuzu[m/10]);//十位
        write_com(0x8f);
        write_data(shuzu[m%10]);//个位
    if(o==1)              //上限选中标志*号可操作
    {
      write_com(0xc0);
      write_data(shuzu[14]);
      write_com(0xca);
      write_data(shuzu[12]);
    }
    else if(o==2)    //下限选中标志*号可操作
    {
        write_com(0xca);
        write_data(shuzu[14]);
        write_com(0xc0);
        write_data(shuzu[12]);
```

```
    }
    else if(o==0)       //隐藏*号不可操作
    {
        write_com(0xca);
        write_data(shuzu[12]);
        write_com(0xc0);
        write_data(shuzu[12]);
    }
    if(u/100==0)       //上限 100 内温度
    {
        write_com(0xc3);
        write_data(shuzu[12]);//空格
        write_com(0xc4);
        write_data(shuzu[u/10]);//十位
        write_com(0xc5);
        write_data(shuzu[u%10]);//个位
    }
    else if (u/100==1)//上限 100 外温度
    {
        write_com(0xc3);
        write_data(shuzu[u/100]);//百位
        write_com(0xc4);
        write_data(shuzu[(u-100)/10]);//十位
        write_com(0xc5);
        write_data(shuzu[u%10]);//个位
    }
  //下限温度
    if (l<0)//下限低于 0 的温度显示
    {
        p=-l;
        write_com(0xcD);
        write_data(shuzu[11]);//负号
        write_com(0xcE);
        write_data(shuzu[p/10]);
        write_com(0xcF);
```

```
        write_data(shuzu[p%10]);
    }
    else if(l>=0)//下限高于 0 的温度显示
    {
        write_com(0xcD);
        write_data(shuzu[12]);
        write_com(0xcE);
        write_data(shuzu[l/10]);
        write_com(0xcF);
        write_data(shuzu[l%10]);
    }
}
void fuhao()   //温度正负子程序
{
        if(f==1) //温度为正
        {
            write_com(0x83);
            write_data(shuzu[12]);
        }
        else if(f==0) //温度为负，显示负号
        {
            write_com(0x83);
            write_data(shuzu[11]);
        }
}
void display(unsigned int date)       //显示温度，
{
        if(date/100==0) //100 度内
        {
            write_com(0x84);
            write_data(shuzu[date/10]);//十位
            write_com(0x85);
            write_data(shuzu[date%10]);//个位
            write_com(0x86);
            write_data(shuzu[13]);//小数点
```

```
            write_com(0x87);
            write_data(shuzu[xiaoshu1]);//小数点后一位
            write_com(0x88);
            write_data(shuzu[xiaoshu2]); //小数点后二位
        }
        else if(date/100==1)       //100 度上
        {
            write_com(0x83);
            write_data(shuzu[date/100]);//百位
            write_com(0x84);
            write_data(shuzu[(date-100)/10]);//十位
            write_com(0x85);
            write_data(shuzu[date%10]);//个位
            write_com(0x86);
            write_data(shuzu[13]);//点
            write_com(0x87);
            write_data(shuzu[xiaoshu1]);//小数点后一位
            write_com(0x88);
            write_data(shuzu[xiaoshu2]);//小数点后二位
        }
}

void key()              //按键加减标尺

{
 //尺度
    if(keym==0)//判定
    {
       delay_50us(100);//延时
       if(keym==0)
       {
          y++;//转换加减的数值
          if(y>=3)y=0;
          switch (y)
          {
```

```
            case 0:m=1;break;
            case 1:m=5 ;break;
            case 2:m=10 ;break;
            }
            while(keym==0);//自锁
        }
    }
  ///换
    if(keyh==0)//上下限转换按键
    {
        delay_50us(100);
        if(keyh==0)
        {
            o++;
            if(o>=3)o=0;//1 为上限选中，2 为下限选中，0 为隐藏
            while(keyh==0);
        }
    }
/////上限
    if(keyu==0&o==1)//上限加按键
    {
        delay_50us(100);
        if(keyu==0&o==1)
        {
            u=u+m; while(keyu==0);
        }
    }
    if(keyl==0&o==1) //上限减按键

    {
        delay_50us(100);
        if(keyl==0&o==1)
        {
            u=u-m; while(keyl==0);
        }
```

```
    }
//// 下限
    if(keyu==0&o==2)//下限加按键
    {
        delay_50us(100);
        if(keyu==0&o==2)
        {
            l=l+m;while(keyu==0);
        }
    }
    if(keyl==0&o==2)//下限减按键
    {
        delay_50us(100);
        if(keyl==0&o==2)
        {
           l=l-m;while(keyl==0);
        }
    }
}

 void   BJ()//温度报警子程序
 {
     if(f==1)    //温度大于 0 的温度比较
       {

                if(sdata>=u) //温度大于上限，报警
                {
                     display(sdata);button();
                }
                else if(l>0&sdata<=l)    //温度小于下限，报警
                {
                    display(sdata);  button();
                }
                else          display(sdata);
       }
```

```
        else if(f==0)  //温度小于 0 的温度比较
        {
                if(l>=0)
                {
                    display(sdata);button();
                }
                else if(l<0)
                {
                    if(sdata>=-l)   //温度于下限，报警
                    {
                        display(sdata);     button();
                    }
                    else if(sdata<-l)   //温度大于下限不报警
                    {
                        display(sdata);
                    }
                }
                else          display(sdata);
        }
    }
   void main()
   {
      buzz=1;
      display0(); //欢迎，
      display1();//框架
      while(1)
      {   display2(); //上下限，尺度
          ReadTemperature();    //测温程序
          fuhao();//负号显示程序
          key();   //按键
          BJ();    //报警程序
      }
}
```